
# The P2 Architect's Guide

*Thinking in Cogs, Pins, and Forces*

August 2026

Version 1.0.3

**Guide Organization**

**From a project idea to a realized build — designing the system, decomposing it onto the P2, and doing it with an AI agent's help.**

**The Three Acts**



**Reference**




Iron Sheep Productions, LLC

P2 Knowledge Base Project

# Contents

# Copyright and License



## Acknowledgments

This guide stands on work done by others:

**Parallax Inc.** for the Propeller 2 microcontroller and the reference documentation that defines its behavior.

**Chip Gracey** for designing the P2 — the eight-cog architecture, the smart pins, the CORDIC solver, and the streamer this guide teaches you to think with.

**The P2 community** whose drivers, projects, and hard-won design habits shaped how this guide frames “thinking in P2.”

## Sources

This guide is a distillation, not a primary source. It draws on, and points you back to, these trusted P2 documents:

- **Getting Started with the Propeller 2** — this guide's companion and **prerequisite**; it teaches the orientation (the chip, and how to read its code) that this guide assumes.
- **The Parallax Propeller 2 Documentation (v35, Rev B/C)** (Chip Gracey, Parallax Inc.) — the architectural ground truth behind the hardware design of Act I and the decomposition of Act II.
- **The P2 reference manuals** (Assembly Language, I/O & Smart Pins, Streamer, DEBUG) — the depth this guide deliberately leaves to them (see *Where to Next*).

# How to Use This Guide

This is a short, narrative guide, not a reference manual — it is meant to be *read*. It assumes you have already met the Propeller 2; if you haven't, its companion **Getting Started with the Propeller 2** is the place to begin. This guide moves in **three acts**, and different readers can enter at different doors:

- **Building a real system?** Read straight through. **Act I** gets the project off the ground — choosing the hardware and buses, spending the pin budget, getting the parts to talk. **Act II** derives the software architecture — which cog owns what, how the pieces talk. **Act III** walks the whole process again with an AI agent at your side.
- **Already have a hardware design and need the software architecture?** Go straight to **Part II** (Chapter 5) — the functional-decomposition method — and use Part I as reference.
- **Curious how an AI agent changes the work? Part III** (Chapters 10–14) revisits every step of the process with an agent in the loop — where it helps, and where judgment stays yours.
- **Coming from the Propeller 1?** Follow the bronze **"P1 note"** sidebars wherever a design decision differs from the P1.

# Conventions

A few conventions run through the whole guide:

- **"cog," never "CPU" or "core."** The P2 community treats a cog as *the computer*, and so do we.
- **Code shows named constants, not raw numbers.** Examples use the compiler's symbolic constants (a pin's name, `_clkfreq`) the way you'd actually write them — and every code example compiles.
- **Code blocks are colored by language** — Spin2 in **blue**, PASM2 (assembly) in **green** — the same IDE-aligned scheme as the rest of the P2 manual family, so code is recognizable at a glance.
- **"P1 note" sidebars** (bronze boxes) are short asides for readers migrating from the Propeller 1, each labeled *same as P1*, *changed in P2*, or *new in P2*. A newcomer can skip every one of them without losing the thread.

- **Inline markers are used sparingly** — 💡 **Tip** for a non-obvious orientation insight, ⚠️ **Watch out** for a genuine pitfall. This is a narrative guide, not a reference peppered with boxes.

# Part I — Getting a Project Off the Ground

Picture a real project — not a toy. Something you're going to build and then put in front of people: a product you'll sell, a job you were contracted to do, a design you'll stand behind. In practice it's usually a blend of those, and the stakes are the same either way — it has to actually work, for someone who isn't you.

Before you can ask the question **Part II** answers — *which cog owns what?* — there's a whole body of work every such project makes you do first. None of it is decomposition yet. All of it shapes the decomposition that follows: by the time you're ready to carve the embedded application into cooperating cogs, this is the work that has handed you the parts, the pin map, the rates, and the deadlines you'll carve *around*. This part is a map of that front end — the things you *do*, and the things you have to *deal with*, to get a project from an idea to a wired-up, understood embedded application.

A word on where this comes from. The projects behind this part were built by an engineer who works mostly by curiosity and request rather than a sales mandate — picking up a thing because it's interesting and seeing how the P2 applies to it. But the work is the same work any shipping project demands, which is why the "you're shipping this" framing holds throughout. And no two projects hit these steps in the same order or the same weight; read for the *shape* of the front end, not for a checklist.

# Chapter 1: Deciding What to Build

Projects start in more ways than a plan admits. One begins as a standing interest — you've always wondered what you'd do with a grid of Hall-effect sensors, and one day a board shows up that has them. One begins as a request — someone hands you a piece of advanced motor-control technology and asks you to make it usable by a whole community. One begins with a thirty-second video of digits morphing into each other on an LED panel, and the thought *I have a driver that could do that.* The trigger doesn't much matter; what matters is that the very next decisions set the project's character, and you make them before you write a line.

The first real one is rarely *how* — it's *is this even practical, and what will it cost me in parts and pins?* Feasibility comes before design. The sharpest version of the question is to ask what the hardware can even do before you build around it. On one imaging project the useful question wasn't "how do I display this data" but "how fast can I even *read* the sensor?" — and the answer, on the order of thirteen hundred frames a second, reshaped everything downstream. Paired with

it is the honest question of what's *practical*: sixty frames a second is already more than an eye needs, so the real design lives in the gap between what's possible and what's worth doing.

Then you choose the peripherals and how you'll talk to them, and a lot of the project's character is decided right here. A device that speaks a narrow, embedded-friendly bus — I²C or SPI — folds into a P2 design cleanly; a device that wants a wide, host-style interface, like a camera ribbon, pulls you toward a different and heavier kind of system. Choose narrow when you can: on the robot dog, trading a Raspberry-Pi camera for a small camera with built-in AI that speaks over I²C kept the whole design embedded and simple. And some parts do the hard work *for* you — a self-contained module you merely *configure* instead of *program*. A trainable voice sensor that just sends an integer for each word it recognizes — so your only job is to receive those integers and translate them into actions — can delete an entire subsystem before you start. Picking one of those is a design decision worth making on purpose.

Sometimes the largest early choice is what the P2 *shouldn't* do at all. The P2 has no operating system and no network stack; when a project needs the web, a filesystem, or the time of day, you reach a fork: port all of that onto the P2, or pair it with a device that already does it well and just talk to it. On one gateway project a Raspberry Pi carried the web server, a mail path, and network time while the P2 did the real-time work, the two lashed together over a couple of megabits of serial — and suddenly the P2 "knew what time it was" without owning a clock. That partition — what runs where — is among the most consequential decisions you'll make, and you make it here, before any cog exists.

Last, scope the features honestly: what will this actually do, and how far will you take it? Text? Graphics? Rotation — portrait, landscape, upside-down? Four digits composed into a clock? The answer bounds everything that follows.

# Chapter 2: Learning the Hardware

Now the part nobody advertises: on a real project, most of the early effort goes into simply finding out *how the parts work.*

Datasheets come first, and they fight you. Some are hard to find; some arrive in a language you don't read and must be translated before they're any use; some don't exist at all, and the nearest thing to documentation is a folder of half-working example code in three different languages. The controller chips behind LED-matrix panels were a running example — new panel, new controller, datasheet nearly impossible to locate and often in Chinese — while an HDMI path had no datasheet at all, only example code close enough to adapt. When there's no schematic *and* no datasheet, you reverse-engineer: read the vendor's example code, watch what it does on the wire, and reconstruct the protocol yourself. Bringing the robot dog over from its original Arduino meant exactly that — no schematics, only example code, every sensor and actuator's communication rebuilt by study.

Then the physical realities the software never mentions. Voltage: the P2's pins run at 3.3 V and plenty of devices want 5 V, so you need level shifters — and if you mean to run the part *fast*, the shifters have to keep up, which quietly narrows your options before you've chosen anything.

Pins. This is the "Pins" the book's title promises, and it's where a design first meets the wall of a finite resource. You count what the part needs, and sometimes it just doesn't fit — too many signals to hand-wire, more than a single adapter's worth of lines. When that happens the answer isn't to drop a feature; it's to build the interface hardware: a small board carrying the level shifters, the connectors, and — if you're wise — a spare row of pins for a logic analyzer. A HUB75 matrix needed more than eight and fewer than sixteen pins per adapter, impossible to hand-wire, so a custom adapter board got designed — and eventually Parallax sold it as a product. Deciding the pin *layout* is a real design act too, and the P2's any-pin-any-function flexibility means you get to make it well rather than accept whatever the package forces.

Some projects need more than a board; they need a part you *fabricate*. A fixture to hold sensors at a fixed geometry, a platform to bolt the controller to the thing it controls. Four time-of-flight sensors aimed to sweep 180° only mean something if a 3D-printed frame holds them at their exact angles; putting the P2 and a voice sensor onto the dog took two rounds of 3D printing. The mechanical build is part of the project whether you planned for it or not.

A special case worth naming: some devices don't run *your* code at all — they run *their own*, which you upload first. What you "download" for such a part is a binary image you load *into* the device; only once it's running its own firmware can you open communications with it. That means building a loader before you can even say hello, as the time-of-flight sensor demanded.

Through all of it, one instrument does the heavy lifting: the logic analyzer. The first question on any new part is blunt — *am I talking to it correctly, and am I getting anything back?* — and the logic analyzer is how you answer it, which is why it pays to design every rig so you can always attach one. Sometimes the rig itself is the test: two of the same chip wired together so each checks

the other, as when an eight-channel serial driver was certified by lashing two P2s together with sixteen wires and verifying every round trip. Underlying all of it is the routine work of tracking where each part plugs in and how it is wired.

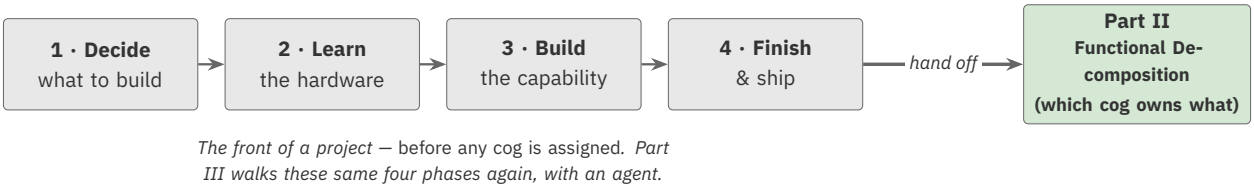



**Figure 2.1.** The front of a project as four phases — decide what to build, learn the hardware, build the capability, finish and ship — handing off into Part II's decomposition. Not one cog is assigned until the hand-off; Part III walks these same four phases again, with an agent.

# Chapter 3: Building the Capability

With the part talking, the work turns to making it *usable* — and here design taste starts to matter.

The interface comes first, and it's more than exposing what the chip does; it's deciding how someone will *think* about the thing. The strongest move is to unify. Study how different communities already reason about the problem, then design one interface a person from any of those backgrounds can pick up and use. The motor-control work reached servos, brushless motors, and wheeled drive through a single interface, so you come to it however you learned motors and still know how to drive it. And a lesson that repays attention: every time you layer a new capability *on top of* your own driver, it tends to *improve the driver itself*, because the new thing needs something you didn't anticipate — adding a morphing-digit display on top of a matrix driver made the driver's own interface richer.

Above the raw interface you add convenience layers that hide the primitives, so the application can say "steer" instead of setting two motor speeds, or treat an animated digit as "just another font." Much of the building, too, is *translation and digestion*: the reference implementation you're learning from is almost always in another language — C, an Arduino sketch, NodeMCU — and you carry it across into Spin2 one idea at a time, often in your head. Sometimes you don't transcribe at all; you write a small program that *generates* what you need, the way a short Python script produced the digit-to-digit transition tables that were then baked into the object.

Then there's the thing that keeps a P2 developer up at night in the best way: *performance.* The reference drivers rarely run as fast as the P2 can, and matching what they do while going faster — and staying error-free at speed — is often the entire point. An eight-port serial (UART) driver pushed past two megabits per second on every port at once, error-free; the matrix driver is a standing chase after the best frame rate the panels will give.

You also *characterize* the hardware — measure how it truly behaves, not how the datasheet claims it should. How wheels behave against a motor's top speed under different batteries; how repeatably a servo returns to a commanded position. And here's the quiet payoff: those measurements don't stay in a lab notebook — they *become the features and the limits of the product.* Characterizing which batteries could drive the motor platform turned directly into its supported-battery spec. Then you verify: checksums, round-trip confirmation, and again the logic analyzer as the court of final appeal, proving the protocol is tight before you call it done.

One honest note before we move on. Sometimes a project meets *your own* limits, not the chip's. A six-axis arm stalled at the edge of one engineer's comfort with the mathematics of inverse kinematics — the code was reachable, the math wasn't, so the demo could pick a thing up and move it but not much more. The opposite happens too: a project you bring deep prior expertise to almost builds itself, the way years of Linux and web experience made the hard parts of a gateway routine. Where your own ceiling sits is part of the real shape of a project — and it's exactly the place **Part III** will have the most to say.

# Chapter 4: Finishing and Shipping

A project isn't done when it runs. On every one of these the same closing ritual runs: *document it so the driver is genuinely usable, post it to the repository in a form people can pick up, and announce that it exists.* SKIP any of the three and you've wasted the work.

Documentation here means more than prose. It's photographs of the actual devices you drove, short videos so people can watch the thing move, and — a signature of real hardware work — the logic-analyzer traces themselves, published as proof of how the communication behaves. If you want the work to outlive the project, you make it *reusable and configurable*: pull the general part out of the specific one, let it be configured per device instead of hard-coded, record which channel each thing lives on. The servo work became a standalone, reusable driver extracted from the arm that first needed it.

And then the long tail the first release never shows you. Vendors ship new firmware and new code; keeping up means diffing their changes against what you built and deciding what to fold back in — a chore heavy enough to stall a project for a year. The time-of-flight driver still carries a known gap, a coordinate table and some angle math left undone, with a pile of newer vendor code waiting to be reconciled. A project can be shipped while honestly incomplete, as long as what isn't finished is documented clearly.

## Where this leaves you

That's the front of a project. You decided what to build and what the P2 should and shouldn't do; you learned the parts, wired them, and proved they talk; you designed an interface, made it fast, characterized it, and shipped it with its documentation. Not one cog has been assigned yet — and that is the point. All of this is the raw material **Part II** works on.

Part II takes exactly this — a wired-up, understood embedded application with a pin map, a set of parts that talk, and a feel for their rates and deadlines — and derives the software architecture from it: which cog owns what, how the pieces talk across the gaps, what adapts between cadences. You may have noticed a few of the hardest questions raised here went deliberately unanswered: two of the same sensor sharing one bus, a fast producer feeding slow displays, a clutch of tiny sensors that don't each deserve a cog of their own. Those aren't front-of-project questions; they're *decomposition* questions, and they belong to Part II.

And a promise to close on. **Part III** comes back to this very list — every phase you just read — and asks it again with an AI agent at your side. Because every one of these things, from hunting down a datasheet in a language you don't read, to reconciling a vendor's new code, to reaching past your own math, changes when you have one.

# Part II — Thinking in P2: Functional Decomposition

You can already write a P2 program. You can launch a cog, drive a pin, share data through hub, and choose Spin2 or PASM2 for a given job. That's the hard part of getting started, and it's behind you. What's left is the part that turns a working program into a *good* design: looking at a whole problem and deciding what goes on which cog in the first place — how to carve the embedded application into the right set of cooperating pieces. You're ready for that now, and this part is about how it's done.

We're going to do something different now. So far — in the material that brought you this far — the work has been to *describe*: the chip, and the system you're building around it. This part *reasons* instead. Functional decomposition — the craft of cutting a system into parts — is a real engineering discipline with decades of literature behind it, and we're going to treat it that way: carefully, and without pretending it's easy. The good news is that the P2 makes the reasoning unusually concrete. On a lot of processors, "how should I structure this?" is a matter of taste. On the P2, as you'll see, the structure is mostly *derived* — the hardware and the timing hand you most of the answer, if you know how to ask.

One thing before we start, and it matters enough that it shapes this whole part: **there is no single right answer that this part can hand you.** Every embedded application is different, so every good decomposition is different. What you can learn — what generalizes — is the *method* for deriving one. So that's what we'll teach: the forces that do the cutting, the order to apply them in, and the way to judge the result. Late in this part we'll watch the whole method run on one example application, start to finish. Read that example to see the moves, never to copy the answer — your application will give a different, equally sound shape.

# Chapter 5: Computing in Space, Not Just in Time

Start with the idea that makes the rest of this part worth the effort. It's the one you first met in *Getting Started*, where each cog just keeps running its own job, independently. Here's where we cash it in.

There are two very different ways a chip can compute. A conventional microcontroller computes **in time**: one core runs a sequence of instructions, one after another, and the way it does more is by

going faster or by slicing that one core into time-shared pieces. An FPGA sits at the opposite pole — it computes **in space**: you lay out function as actual parallel hardware, many things happening physically at once, with no single instruction stream at all. These aren't just two speeds; they're two fundamentally different shapes of computation.

The Propeller 2 lives between them, and closer to the spatial side than you might expect. Its eight independent, deterministic cogs and its sixty-four programmable smart pins form what's best described as a **coarse-grained spatial fabric**: not the fine-grained sea of logic gates an FPGA gives you, but a modest number of real, parallel computing elements you can assign function to, each running its one job continuously. Decomposed well, a P2 design behaves like spatial hardware — parallel pipelines whose throughput is set by the *rate* data flows, not by how many instructions any one stage runs. Decomposed badly, the very same silicon collapses back into a slow sequential machine: one cog doing everything in turn while the other seven idle.

That sentence is the reason this part exists. The whole discipline of P2 decomposition is, at bottom, the practice of keeping your design on the *spatial* side of that line — of spreading function across the fabric instead of funnelling it back through a single core out of habit. Everything that follows is in service of that.

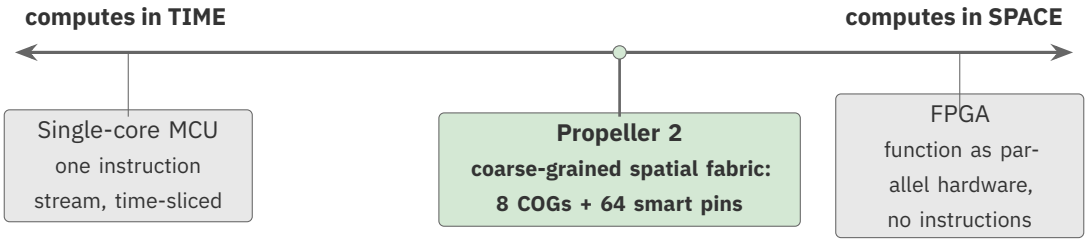



**Figure 5.1.** Computing in time vs. in space. A single-core microcontroller runs one instruction stream; an FPGA lays function out as parallel hardware. The Propeller 2 sits between them — a coarse-grained spatial fabric you assign function to.

**P1 NOTE**

**The idea is old, the room is new.** If you've built P1 designs, you've been thinking spatially all along: dedicating a cog to a job and letting it run is exactly this mindset, and the original Propeller pioneered it. What the P2 changes is how *much* fabric you have to lay function onto. Smart pins can now absorb an entire bit-banged protocol that used to cost you a whole cog; the CORDIC and the streamer take on work that used to live in cog code; the hub is sixteen times larger. The instinct transfers intact — you have far more space to spread into, and more reason to.

The deep treatment of this space-versus-time framing — including an honest account of what the P2 borrows from FPGA thinking and, just as importantly, what it *doesn't* — is in Appendix A. Here, the thesis is enough: **the P2 computes in space when you let it, and decomposition is how you let it.**

# Chapter 6: Where Object Shape Comes From

Here's the central move of this whole part, stated plainly: on the P2, the shape of your object set is not a matter of taste picked from a menu. It is *derived* by reconciling a small number of physical and architectural forces. Change the buses, the deadlines, or the data rates, and the correct object set changes with them. A good decomposition is therefore an *answer to constraints*, not a style choice.

That distinction has a practical payoff. If decompositions were chosen by taste, the most you could do is collect examples and imitate the nearest one. Because they're derived, you can learn the forces that do the deriving — and then produce a sound design for an embedded application you have never seen before, because you're reasoning from its wiring rather than pattern-matching to something you saw once. Reasoning from the forces generalizes; copying an example doesn't.

It helps to separate two things that are easy to blur together. The **objects** you can build with — a top-level application, a device driver, a semantic driver, a policy layer, a buffer, a coordinator — are your *vocabulary*: the nouns. The **forces** are the *grammar*: the rules that decide which nouns to instantiate, how many of each, and where the boundaries between them fall. A vocabulary list tells you what words exist; only a grammar tells you how to build a correct sentence you've never spoken before. This part is about the grammar. (The vocabulary — the object archetypes — we'll lean on as we go rather than catalogue here.)

## Two axes, co-designed

Classical software decomposition works on a single axis, and it's a purely logical one: split behavior into modules, and judge the cuts by **cohesion** (do the things inside a module truly belong together?) and **coupling** (how much has to cross between modules?). That axis is real and we'll use it — it rests on decades of solid work, which Appendix B points you to.

The P2 adds a *second* axis, a physical one: **allocation onto a finite, heterogeneous resource lattice** — eight cogs, sixty-four smart pins, one shared CORDIC, sixteen locks, a bounded amount of hub bandwidth, adjacent-cog LUT sharing, the streamer. And here is the insight that makes the P2 special to design for: *that physical axis is not only a constraint — it's a decomposition tool in its own right.* A cog is the strongest encapsulation boundary the silicon offers: private memory, deterministic timing, no interference from its neighbors. A smart pin can *delete an entire software module* by absorbing its function into hardware. So "where does this boundary go?" and "what hardware runs it?" are not two questions asked in sequence — they're one decision, made together.

The two axes are co-designed, and they keep each other honest. A boundary chosen on the logical axis that ignores the lattice gives you an elegant module that can't actually run — no cog free

to host it, or the hub saturated feeding it. A boundary chosen on the physical axis that ignores cohesion gives you a cog that owns three unrelated jobs and is impossible to test. You reconcile both. When they genuinely conflict, the resource budget — an artifact we'll build later in this part — is what decides.

## The failure this prevents

It's worth naming the mistake all of this exists to prevent, because it's the natural thing to do and it looks fine right up until it doesn't. Call it the **flat device list**: every chip gets a driver, every driver is a sibling reachable from `main()`, and the shape was chosen by analogy to some example rather than derived from how the hardware is actually wired. It compiles. It even runs during single-cog bring-up. Then it fails — as intermittent, timing-dependent, nearly-undebuggable flakiness — the first moment the derivation it skipped would have forbidden the cut. We'll see exactly how that happens when we meet Force 1. The cure is to derive the shape from the wiring instead of guessing it; the four forces are how you do that.

# Chapter 7: The Forces That Do the Cutting

Four forces do the work. Three of them are **primary** — they cut the object set horizontally, deciding who owns what and how the pieces relate. The fourth is **emergent**: it falls out vertically, once the first three have drawn the structure. We'll take them one at a time, and we'll lead each with the *question it asks*, because that question — asked of your own embedded application — is the technique you're meant to carry away. The robot dog and the I²C buses you'll see are illustrations of a force in motion, never a rule to transplant.

A word on emphasis before we start: for each force, the *why* matters more than the *what*. An engineer who knows why a force exists on the P2 specifically will generalize it to hardware we never imagined; one who memorizes a rule will eventually meet the case the rule didn't cover and apply it wrongly. The sections below therefore lead with the reasoning, not the rule.

## Force 1 — Who owns this wire?

The first force asks a **correctness** question, not a style one. Of the four, it's the only one that can make your program flatly *wrong* rather than merely inelegant, so it goes first.

The question is: for each serialized, stateful hardware resource — an I²C bus, a one-wire LED chain, a smart pin in the middle of a transaction — *which single cog owns it?* And the answer the force insists on is: exactly one. One owner per resource, and the object boundary traces the **wire**, not your feature list.

The reason is physical, and it comes straight out of the silicon. P2 pin outputs are OR'd together — there is no hardware referee arbitrating who gets the pin. If two cogs both drive the same SDA and SCL lines, they don't take polite turns; their outputs combine, and a bus transaction — which is a multi-step sequence (start, address, acknowledge, data, stop) that assumes a single agent in charge — is corrupted. This isn't "a race you might lose." A bus is a stateful protocol, and a stateful protocol with two uncoordinated drivers is *guaranteed* to break. The chip gives you sixteen locks and atomic single-long hub access to coordinate shared *data* — but a lock can't un-corrupt a half-issued I²C frame. The clean coordination is therefore *structural*: make the resource un-shareable by giving it a single owning object in a single cog. The hardware's lack of a referee is precisely *why* ownership has to be explicit and singular in your software.

So Force 1 makes the primary cut, and it's usually a cog boundary. Group the devices by which wire they sit on and what timing that wire has to meet; give each group one owning cog and one transport object. And notice what decides the *shape* of that transport — it's the sharing topology, not the protocol. Several devices sharing one bus inside one cog want a single shared transport with one configuration that the device drivers call into. One device alone on its own bus wants a self-contained transport with nothing to coordinate. You can end up with the *same protocol*

*implemented twice with two different state models* — and that's correct, because how many things share the wire, not which protocol it is, decided the shape.

⚠ **Watch out:** the flat device list is this force ignored. The moment two cogs touch one bus, you get silent corruption that presents as flaky hardware — intermittent, timing- dependent, and miserable to debug from the symptom, because the symptom is three layers away from the cause. A design that picks its shape from *how many devices exist* rather than *who shares a wire* has this failure built in from the start.

**P1 NOTE**

**Same as P1, and just as strict.** Single ownership of a serialized resource was already the rule on the P1, for the same reason: its pins, too, gave you no hardware arbiter. If you internalized "one cog owns the bus" on the P1, that instinct is exactly right here — the P2 hasn't relaxed it. What the P2 adds (next force but one) is a way to move some of those resources off cogs entirely.

### One owner, several shapes

"One owner per resource" is the rule, but the *shape* that owner takes depends on who needs the wire — and picking the shape is part of the cut. Three shapes cover almost everything:

- **One cog, many callers.** Several device drivers inside one cog share a single transport object — the shared bus-1 singleton you'll meet in the worked example. The cog is the owner; the drivers call in.
- **One bus, many *cogs*.** Sometimes several *cogs* need the same bus — a PSRAM framebuffer, an SD card. You can't hand a bus back and forth between cogs: the P2 has no hardware bus arbiter and no cog-independent DMA (the streamer and FIFO each belong to one cog). So a resident **broker cog** owns the bus, and every other cog posts its request into a private mailbox slot the broker services in turn. The owning cog *is* the arbiter.
- **The same bus, replicated.** Two identical I²C trees, say. The trap here is believing "one owner" forces one shared copy of the driver code — it doesn't. *One owner per bus* is the correctness rule; a single shared code image is merely its cheapest **encoding** when there's exactly one bus. With two, give each bus its own state — index the state by bus, or hand each its own context — and keep exactly one writer per bus. The rule holds; only the encoding changed.

The through-line is worth carrying away: don't confuse the *rule* (one owner) with the cheapest *encoding* of it (a single shared image). Keep the rule; let the sharing topology pick the shape. The P2 coordination mechanisms themselves (locks, atomic access, cog attention) are in the *P2 Assembly Language Reference*.

## Force 2 — What does each seam promise?

Once Force 1 has scattered work across several cogs, those cogs have to exchange data. The second force asks: for each place where two cogs meet — each *seam* — *what does the exchange promise?* Does the sender wait for the receiver? Does the receiver always see the freshest value, or every value? Who depends on whom?

That promise is called the **contract** for the seam, and choosing it *is* a decomposition decision, because the coupling you can tolerate determines where the boundary goes. A few contracts you'll reach for: a *blocking call*, where the caller waits on the callee's worst-case latency (tight coupling); a *latest-wins mailbox*, a single slot where the producer never waits and the consumer always reads the newest value (decoupled completely); a *ring buffer*, which decouples the two rates while preserving every sample; *published telemetry*, where one writer puts values in hub and any number of readers take them with no lock at all; and — when one producer feeds *many* consumers with bulk frames rather than single values — *fan-out publication*, a shared pool of frames with one queue per consumer, each reader taking frames at its own pace. Each contract names a different dependency direction, and choosing it draws the boundary.

There's a design *stance* hiding in that second contract, worth pulling out because it shapes how a system *feels*. For a sensor, the responsive move is almost always to **read it continuously and publish its latest value**, and have every consumer take that last-posted value from the mailbox — never to reach out and read the sensor at the moment a value is needed. Reading on demand couples the consumer to the sensor's read latency: the loop that wanted the number *now* waits for a conversion. Reading the last posted value instead is instant and never blocks, and the sensor's own cadence — fast or slow — stops mattering to anyone downstream. The cost is a value that may be a cadence old; for most control and display work that staleness is invisible, and the responsiveness you buy is not.

One of these carries a caution the others don't. Most contracts are cheap to change after the fact — swap a latest-wins slot for a ring buffer later and little else moves. Fan-out publication is the exception: whether the consumers *share* one copy of each frame (a reference count) or each gets *its own* copy reaches into every place a frame is committed, released, and bounds-checked, so changing it later touches every seam. Decide it at derivation time, before the first consumer ships — it's the one contract you can't cheaply walk back.

Why is this a *design* act on the P2 rather than a detail? Because the P2 has no operating system underneath you — no message queue, no IPC layer, nothing imposing a coordination mechanism. Inter-cog coordination is whatever *you* build out of hub RAM, atomic single-long access, the sixteen locks, and cog-attention signalling. That absence is a feature: it means you choose the exact coupling your timing budget allows, with nothing forced on you. An engineer who thinks "there's no free message queue here — I am *choosing* the coupling" designs the seam deliberately. One who reaches for a blocking call out of habit quietly throws away the determinism the chip just gave them, by making a fast loop wait on a slow one.

### One seam, three planes

Here's the part that sharpens Force 2 from a single choice into a real tool. Every seam between two cogs is really *three* relationships superimposed, and each wants its own mechanism:

- The **data plane** — bulk, rate-defined movement. Its concerns are throughput, buffering, and back-pressure; its tools are the streamer, the hub FIFO, burst transfers. Get it wrong and you *waste bandwidth* — visible, and recoverable.
- The **control plane** — commands and state. Low-rate but correctness-critical: atomicity, ordering, who is allowed to write what. Its tools are hub mailboxes, locks, single-writer ownership of each shared long. Get it wrong and you *corrupt state* — an intermittent race.
- The **event plane** — signalling and urgency. Its concerns are latency and priority; its tools are cog-attention signalling, the event system, or deliberate polling. Get it wrong and you *miss a deadline* — and that one stays silent until the field.

Notice they're ranked by the cost of getting them wrong, and you spend your design care in the inverse order: an event-plane mistake is the most expensive and the hardest to see, so it deserves the most thought. The signature way to use the chip badly is to build all three planes on one mechanism — polling a hub flag (a control-plane tool) to deliver an urgent event (an event-plane need), or pushing bulk data through mailbox words (control-plane) instead of the streaming path (data-plane). Naming the three planes is what lets you catch that conflation in your own design before it ships.

There's one small discipline from the control plane worth carrying away by name, because it recurs everywhere on the P2: when you publish a multi-field update through hub, write the payload first and bump the signalling counter *last*. Because a single-long write is atomic, a reader that watches that counter can never catch a torn, half-written value — the publish-last ordering makes a lockless hand-off safe. It costs nothing and it removes a whole category of glitch.

The failure modes Force 2 prevents are two: blocking calls between cogs that quietly *serialize* a system that was meant to run in parallel, and multi-long structures written by one cog and read mid-update by another, producing torn reads that look like glitches. Both fixes are structural — choose the contract deliberately, and publish atomically. The deep treatment of inter-cog contracts and the coordination primitives is in the *P2 Assembly Language Reference*.

## Force 3 — Where do two cadences meet?

The third force is the one a beginner's instinct most often misses, because it corresponds to nothing you can point at. There's no chip for it and no line item in a parts list.

Devices live in different **time domains**. An LED chain wants nanosecond-precise bit timing; a set of servos wants a smooth fifty-hertz stream; a voice recognizer is polled lazily and stretches the clock when it feels like it; a battery reading is meaningful about once a second; an ultrasonic echo is a one-shot event that happens when it happens. The question Force 3 asks is: *where does data cross from one cadence to another* — and what has to sit at that crossing to reconcile the rates?

Because whenever data crosses a cadence boundary, *something must adapt the rate*, and that adapter is a distinct responsibility — so it's a distinct object. The P2 positively encourages you to put different time domains on different cogs and smart pins; that's what eight deterministic cores and sixty-four autonomous pins are *for*. But the instant you do, you've created the software equivalent of a clock-domain crossing — the same problem hardware engineers handle deliberately at the boundary between two clocks — and, like its hardware namesake, it produces glitches if you don't handle it on purpose. (The literature has an exact name for a chip shaped like this — *globally asynchronous, locally synchronous* — and Appendix B points you to it.)

Two kinds of adapter fall out, and they're worth telling apart:

- A **sampler or buffer**, where a fast producer and a slow consumer meet. The rule for picking which is a question about the consumer: does it need *every* sample, or only the *freshest*? Every sample means a buffer; only the freshest means a latest-wins slot. That choice is the whole design of the adapter.
- A **slew or easing engine**, where a discrete intent has to become a continuous stream. A command like "stand" or "walk" is a *step* — it arrives once. A servo physically cannot take a step; it needs a smooth, accelerated-then-decelerated trajectory at its own frame rate. The thing that turns the one into the other — the *ramp* — is a responsibility distinct from both the policy that knows *what* to do and the driver that knows *how* to talk to the chip. Pulling the ramp out of both is what keeps both of them clean.

There's a placement question hiding inside the sampler, and it's worth surfacing because it's easy to get lazily wrong. When one fast producer feeds several consumers at *different* reduced rates — a sensor sampled at full speed but shown on a display at sixty a second and logged at one a second — where does the rate-reduction actually live? You can **fold** it into the producer as a plain one-in-N SKIP: cheap, no extra object, but the decision then happens invisibly, buried inside a commit. Or you can **promote** it to its own visible stage — which you'll want when the reduction is more than a SKIP (averaging, peak-hold, an adaptive rate), or when you simply need to *watch* the decision at runtime. That second reason is a real one, and it's the first hint of a judging tool we'll sharpen later: a cut can be correct and still be one you can't observe.

And there's a third situation that belongs to this force, where it collides with Force 1 in a way worth seeing. Suppose several devices share *one* bus but want *different* cadences — servos at fifty hertz, an IMU at a hundred, a battery at one. Force 3 says "different cadences want separating," but Force 1 flatly forbids splitting the bus across cogs. They can't both win by cutting. The resolution isn't a second cog on the bus — it's **cooperative tasks within the single owning cog**: several small routines sharing that one cog and that one bus, each running at its own cadence and yielding at transaction boundaries so the bus stays coherent. That's a first-class decomposition tool for "shared resource, multiple rates," and it's the kind of answer you only find by holding two forces in tension instead of applying one in isolation.

⚠️ **Watch out:** ignore the rate adapters and you get two classic embedded bugs. Skip the sampler and a slow consumer back-pressures a fast producer (or a fast producer floods a slow consumer) — dropped frames, stalls, torn state. Skip the slew and your servos *snap* to position instead of

moving, drawing current spikes and mechanical shock, because a step went straight to the actuator with no ramp between intent and motion.

**P1 NOTE**

**New room to cross into.** Rate adaptation was always a concern, but the P2 hands you far more places to put a time domain — sixty-four smart pins that each hold their own cadence autonomously, where the P1 had thirty-two plain pins and often a spare cog pressed into bit-banging. That's a gift, but it's also *more cadence boundaries to cross*: every time you push a job out to a smart pin, you've created a crossing back to the cog that needs an adapter. The fabric got wider; mind the seams between its cells.

The implementation patterns for samplers, easing engines, and cooperative tasking live in the Spin2 pattern library; the smart-pin modes that let a pin hold its own time domain are in the *I/O & Smart Pins User Guide*.

## Force 4 — How high does each piece sit?

The first three forces are horizontal: they decide which cog owns what, and how the pieces talk across the gaps. The fourth is the *vertical* consequence that falls out once they've drawn the structure — which is why we call it emergent rather than primary. It answers the question every programmer eventually asks: *how much code goes in one object?*

The honest answer is not a line count and not a component count. It's this: **split where the unit changes, or where the axis of change changes.** Stack the objects within an ownership domain so that each tier does exactly one unit conversion and changes for exactly one reason. The canonical stack climbs from *bits on a wire*, to *device registers*, to *physical units* (millimeters, degrees, millivolts), to *behavior*. Each tier speaks a different unit than the one below it, and that change of unit is the seam.

The principle underneath is an old and durable one — Parnas's *information hiding*: decompose around the things that change independently, not around processing steps. Two pieces of code that will *always* change together for the same reason belong in one object. Two that change for different reasons — a new chip versus a new behavior — belong in different objects, even when they sit in the same CALL chain. A line-count rule would never produce a clean four-tier device stack; the unit-conversion rule produces it automatically, because each unit boundary is exactly a place where the code above and below it change for different reasons.

On the P2 this force negotiates against a hard limit, and you should know it's there: cog- local memory is *tiny* — 512 longs of register RAM, of which 496 are usable for PASM code and data. Unlimited layering isn't free; each tier boundary costs a call and a little state. So the default is one tier per unit conversion, with an explicit escape: when a cog is genuinely tight on memory, fold two adjacent tiers together — but say so, and never fold two tiers that change for *different* reasons just to save space, because that quietly rebuilds the monolith you were avoiding.

That monolith — or worse, a "driver" that mixes register pokes with behavior logic, so that swapping the IMU chip forces you to re-test the walk cycle — is the failure Force 4 prevents. When tiers that change for different reasons are fused, every change ripples across unrelated concerns, and the clean place you *would* have tested at is gone.

## Reconciling the forces

Here's the thing the four-forces list can hide: the real skill isn't applying each force, it's *reconciling* them, because they pull against each other and against plain simplicity. You've already seen one tension — Force 1 says "one cog per bus," Force 3 says "different cadences want separating," and when three cadences share one bus, the resolution is cooperative tasks inside the one owner. There are more like it. Force 2's instinct to decouple every seam reconciles against simplicity — not every hand-off needs a ring buffer; a latest-wins slot is usually enough. Force 4's instinct to layer everything reconciles against that tiny cog memory — deep stacks cost RAM and per-call overhead you may not have.

None of these tensions has a formula. What you do is hold the forces together, let them argue, and let the *hardware and the hardest deadline win* — those are the two things you can't negotiate with. That habit of reconciliation, more than any single rule, is what separates a design that fits the chip from one that fights it.

# Chapter 8: Completing and Judging a Decomposition

## The objects that guard the whole application

The four forces build a clean structural tree: who owns what, how the branches talk, what adapts between cadences, how deep each branch layers. But a real embedded application needs some objects that don't live *in* that tree — they live *across* it. They're driven by concerns that don't respect the ownership hierarchy, and if you only ever apply Forces 1–4, you end up with a tidy tree and nowhere to put the supervisor, the translator, or the calibration data. Naming these cross-cutting concerns is what keeps them from getting smeared across everything. There are five that recur:

- **A safety override.** Some authority has to be able to override the whole application — a low-battery cutoff, a watchdog, an emergency stop — and a fault in one place has to be contained so it can't cascade. This wants an explicit, privileged supervisor sitting *above* the policy layer, able to suppress it.
- **An external-interface translator.** When you integrate a subsystem that has its *own* vocabulary — a sensor's command codes, a vendor's frame format — put a translation object at the boundary so that external naming never leaks inward. The outside vocabulary changes on someone else's schedule; quarantine it behind one seam and a vendor change touches one object instead of your whole codebase.
- **A configuration store.** Separate what varies *per physical unit* — trim offsets, pin maps, per-board personality — from what's fixed *by design*. Identical firmware should run on every unit you build; the per-unit constants belong in data, not sprinkled through your drivers.
- **Testability seams.** Shape the objects so each one can be exercised *standalone on real hardware* before the whole is assembled. On embedded work you can't single-step a servo; you bring hardware up one layer at a time. The seam you can test at is the seam you should cut at — and the need to observe a layer often reveals a boundary you'd otherwise have fused.
- **A lifecycle sequencer.** Objects have a *temporal* dependency graph: power and rails before buses, a chip awake before you actuate it, cogs launched in a safe order. Someone has to own that sequence.

There's a reason several of these have to be *explicit* on the P2 specifically rather than emergent. Cogs are independent — which is wonderful, because a hung cog won't drag the others down, but also means a hung cog won't stop driving its pins on its own, and means init ordering *isn't* implied by your CALL structure the way it is in a single-threaded program, because cogs launch concurrently. The chip gives you deterministic, isolated cores; these cross-cutting objects are how you reimpose whole-application guarantees — safety, ordering, calibration — back on top of that isolation. You can't assume they'll fall out of the design. You place them on purpose, and

you place them *after* the structural tree is drawn, because where each one goes depends on the tree it's guarding.

💡 **Tip:** when you think you're done, go down this list of five and ask "where does each of these live in my design?" — and if one genuinely isn't needed (no external vocabulary, so no translator), say so out loud. An omission you *named* is a decision; an omission you didn't notice is a bug waiting in the field.

## Keeping a budget

Everything so far has been about drawing boundaries. This section is about a number that tells you when you've drawn them wrong.

The P2's resource lattice is *finite*, and you should treat that as a design invariant rather than a thing you discover at the end. There are eight cogs, sixty-four smart pins, sixteen locks, one shared CORDIC, a bounded hub bandwidth, LUT sharing only between adjacent cog pairs, and 512 longs of memory per cog. None of those is negotiable. So a useful habit is to keep a **resource budget** — an allocation table you fill in *as you derive*, not a report you write afterward — listing which cog owns which timing domain, which pins run which mode, where each lock goes, how much hub traffic you're generating, and how much of each finite thing remains. A blank row in that table is a resource you forgot to account for.

The budget earns its keep through one sharp signal. **"Running out of cogs" is the P2's concrete way of telling you the design is too *coupled*.** When the lattice can't hold your proposed allocation, the boundaries are wrong — not the chip. So when you run short, the move is to *re-cut, not cram*: look for a funnel cog that's quietly doing several jobs, a protocol a smart pin could absorb to free a cog, or a seam whose coupling is so high it shouldn't have been cut where it was. There's an honest escape — when every cog genuinely owns one irreducible real-time job and nothing can be absorbed, the design is at capacity, and the answer is to reduce *scope* or move a concern off-chip, not to time-slice a real-time job onto a shared cog. But reach for that escape last, after you've tried to re-cut. Most "out of cogs" is too-coupled in disguise.

That signal only fires once you've run out. There's a complementary check that catches trouble *before* you do: **every cog you assign should have a one-sentence reason it must exist, drawn from a short closed list — determinism, resource ownership, blocking I/O, or throughput.** If you can't say in one sentence why a job needs its *own* cog from that list, it probably doesn't — fold it into an existing owner. Where "out of cogs" catches a design that's too *coupled*, this catches one that's too *inflated*, quietly spending cogs it never needed. A healthy design often ships with a cog or two deliberately in reserve, each cog in use carrying its one forcing sentence — a stronger place to be than having filled all eight by reflex.

# Judging the cut

You can now *propose* a decomposition. The last piece of the method is how to *judge* one — to look at two candidate cuts and say, with more than a feeling, which is better. This is the part most worth slowing down for, because it turns "that seems cleaner" into something you can actually check.

Four tools, in increasing sharpness:

**Coupling, as a countable integer.** On the P2, the coupling between two cogs is physical and *countable* — it stops being a vibe. Across any boundary you draw, count the longs that cross it per unit time, the fields that share an invariant (data that must change together to stay correct), and the locks held across the cut. Minimize that number. Two candidate cuts can be compared directly by their counts, and the lower one wins unless cohesion argues otherwise.

**Change-coupling — the sharpest tool.** The second tool sharpens the first: rather than *count* what crosses, ask what must *change together*. Two pieces of code are **change-coupled** — the design literature calls this *connascence* — if changing one forces a change in the other to stay correct. It comes in *static* forms, visible right in the source (two sides agreeing on a name, a type, a field order), and *dynamic* forms, true only at runtime (two sides agreeing on execution order, on timing, on a value relationship). The governing rule is: maximize change-coupling *inside* a boundary, minimize what *crosses* it, and *convert* the strong dynamic forms into weak static ones right at the seam. On the P2 the dangerous case is specific and worth memorizing: **dynamic change-coupling that crosses a cog boundary** — a timing assumption, an execution-order assumption, a shared runtime value — because the hardware will faithfully express it as *jitter and races*. The publish-last discipline from Force 2 is exactly this conversion in action: it takes a dynamic execution-order dependency between two cogs and makes it safe by construction.

The most common everyday face of change-coupling is plain **duplication**: two objects that compute the same value, or each carry their own copy of the same algorithm. That is change-coupling of the worst kind — two places that must now change together forever, and the day they silently drift is a bug you'll pay for in the field. The fix is a *decomposition* fix, not a coding tidy-up: give the value or the algorithm a **single owner**, and let everyone else reach it through that owner — reading it through an accessor that hides *how* it's composed, so the composition can change in one place without touching a caller. Duplicated logic sitting across a boundary is a reliable signal that the boundary is in the wrong place; the cure is to move the shared thing to one side and let the other CALL in.

**Back-pressure, as a min-cut.** Put the two together and you can state precisely what a good boundary *is*. Every boundary carries a back-pressure equal to the change-coupling forced to cross it times the cost of the channel that carries it — and on the P2 the channel cost is concrete (a mailbox's hub traffic, a lock's contention, an attention signal's latency). A good boundary is a **min-cut**: the cohesion you gain inside each piece exceeds the back-pressure across the cut. That gives you a crisp objective to aim at instead of an aesthetic — draw the boundary where the things that must stay together stay together, and the least, weakest change-coupling crosses the cheapest

channel. If a cut isn't a min-cut, that's your signal that one of the forces placed a boundary wrong, and you redraw it.

**Observability — can you watch it work?** The first three tools all optimize for *correctness* and *cost*; none of them can tell you that a cut, however clean, is one you'll never be able to *see* running. That's a fourth axis, and on the P2 it's nearly free to have: once a boundary is drawn, ask whether each side's decisions can be surfaced at runtime — in a `DEBUG()` on live hardware, in production — *without* reaching inside a critical section to do it. A decision folded into an atomic commit is invisible; the same decision kept as its own small stage can be watched. (This is the judging tool the fold-versus-promote choice back in Force 3 was pointing at.)

There's a P2-specific gift in this fourth tool. Because a seam can publish its state lock-free (Force 2's published telemetry), you can aim a *separate cog* at it purely to watch — an observer that reads the same longs the consumer reads and never writes back. On most machines, instrumenting a live path *perturbs* it: the watcher steals cycles from the very thing it's watching, so you end up measuring a system you changed by measuring it. Here the observer runs on its *own* cog and its reads don't contend, so it slows neither the producer nor the consumer — you **observe without perturbing**. And there's no rule of one: several cogs can watch the same seam at once, because lockless reads don't compete for it. Designing a seam so it can carry a silent bystander — or several — is, on this chip, nearly free, and worth doing on purpose.

Keep it distinct from the testability seam we met earlier: that one asked *can I bring this layer up standalone before assembly?* — this one asks *can I watch this seam's live decisions after it ships?* A cut can pass the first and fail the second, and a real design has un-folded a perfectly testable decimator back into a visible stage for exactly this reason.

These four — coupling, change-coupling, back-pressure, observability — rest partly on a body of design literature older than the P2 and partly on hard-won P2 practice; Appendix B names the sources so you can go deeper when a problem outgrows this part.

## A decomposition is revisable — expect to dial it in

The four tools help you *judge* a cut, but they carry one honest caveat: a decomposition is not *right* simply because you balanced the forces carefully the first time. The forces give you a sound *starting* structure, derived from what you knew when you drew it — and you rarely know everything then. As the system actually comes together, you'll see things the derivation couldn't: a seam that looked clean on paper turns awkward in the code, a side effect surfaces that no force predicted, a cadence you estimated proves faster or slower on real silicon. Any of those is reason enough to go back and *re-balance* — to change how you resolved one of the forces, redraw a single boundary, or move a job to a different cog.

Expect this, and read it as the method working rather than failing. A force balance is a *hypothesis* about the hardware and the deadlines; building the thing is how you test it, and when the evidence contradicts the hypothesis you re-derive *that one decision* against what you now know — re-running the relevant tools on the new fact, not the whole procedure from scratch. The goal was

never to balance everything once and freeze it; it is to converge on a shape that survives contact with the running system. Dialing one in over a few passes is normal, and every pass is anchored by the same forces.

Experience shifts the odds. The more decompositions you've derived, the fewer times you'll have to rip one up and lay it down again — you learn to see the awkward seam or the hidden cadence *before* it reaches code. But no amount of experience closes the door entirely: a genuinely new element can still hand you a fact your instinct hadn't met, and force a rethink. That isn't a failure of skill; it's the nature of designing against real hardware. (The retrospective form of this discipline — comparing what you *derived* against what you actually *built*, once the code ships — is the as-built audit in the next chapter.)

# Chapter 9: The Method in Action

## The first-contact procedure

We now have the forces, the cross-cutting objects, the budget, and the way to judge a result. The last thing you need is the *order* to apply them in — because the forces are orthogonal, but the work isn't: some choices depend on earlier ones (you can't pick a seam's contract before you know where the cog boundaries are). Here is the routine to run the first time you meet a hardware mix, before you write a single object. Think of it as a method you *adapt*, not a script you obey — the spine steps always run, and the others state when you can skip them.

The procedure deliberately *inverts* the classic top-down approach. You don't start from the data model. You start from the **hardware edge and the timing budget**, and let the structure fall out of them:

1. **Enumerate the wires.** What buses, timing-critical pins, and discrete signals exist? List the serialized resources. *(Always runs — everything downstream depends on it.)*
2. **Triage against the smart pins.** For each peripheral, can a smart pin absorb its protocol entirely in hardware — PWM, serial, quadrature, an ADC or DAC, edge counting? The ones a pin can own drop out of the cog-cadence problem completely. This is the physical axis used as a tool: a smart pin *deletes* a software module. *(Skip for a protocol no pin mode covers — a multi-byte I²C transaction stays a software-owned resource.)*
3. **Assign owners.** Group the survivors by bus and timing budget, and give each group exactly one owning cog and one transport object. Let the sharing topology pick the transport's shape — shared singleton for a shared bus, self-contained instance for a sole device. *(Always runs — this cog map gates every later choice.)*
4. **List the cadences.** At what rate does each device want service, and where do two rates meet? This surfaces the rate-domain boundaries and the discrete-to-continuous paths. *(Skip only if everything runs at one shared cadence with no easing path.)*
5. **Resolve same-bus rate conflicts.** Is any single bus serving multiple cadences? If so, the answer is cooperative tasks *within* the owning cog, not a second cog on the bus. *(Skip when no bus serves multiple cadences.)*
6. **Draw the seams.** For each inter-cog edge, what coupling does the deadline allow — and design its data, control, and event planes separately. *(Skip for a single-cog design with no seams.)*
7. **Layer each branch.** Within each ownership domain, how many distinct unit conversions are there? One tier each. *(Collapse tiers where cog memory is tight — and say so.)*
8. **Place the cross-cutting objects.** Where do the safety override, the translator, the configuration, the test seams, and the sequencer go? *(Name the ones a given application doesn't need, so the omission is a decision.)*
9. **Reconcile.** Where do two forces disagree, and does the result fit the budget? *(Always runs — the reconciliation against the hardest deadline is what makes the output sound.)*

One more property worth knowing: the procedure is **fractal**. After the top-level pass, you can run the very same routine *inside* a cog that owns a bus — it has its own internal cadences, its own seams between cooperative tasks, its own layers. Apply it at whatever altitude you're working.

When you're done, you hold two things: the object-and-cog set, and the resource budget that proves it fits. Judge it with the four tools from the last section before you commit a line of code.

One more practice belongs to the method even though it runs *after* the code ships — because it's what keeps the method honest. Once the application is running, go back and compare the decomposition you *derived* against the one you actually *built*: which cuts survived to code, and which quietly changed. Tag each divergence by the kind of reasoning behind the original cut — a hard hardware-or-timing fact, a durable principle, or a softer heuristic. The pattern is reliable enough to count on: cuts anchored in the hardware survive, cuts anchored in a pattern you merely liked tend to drift. That comparison — an **as-built audit** — tells you which of your forces were truly load-bearing, and it catches the one thing a forward derivation never can: the mechanism you *built but only half-adopted* (a wake path wired into a buffer manager but used by only one of three consumers, say). Catching those quiet divergences is what the as-built audit is for.

## Watching the method run: a walking robot

Let's watch the whole method run, once, end to end, on a single application — a small walking robot, a quadruped "dog." Before we start, the one thing that matters most about this section: **this is one application's answer, shown to make the method visible — it is not a template.** Your application will be different, so the object set you derive will be different. Read for the *moves* — which force fires at each step and why — never for the result. If you ever catch yourself copying a boundary from here into a design of your own, stop, and run the procedure against *your* wiring instead. That's the whole point of having a method rather than a catalogue.

Here's the only input we start from — the hardware, nothing else:

- **I²C bus 1** carries a multi-channel servo/PWM controller — driving **thirteen servos** (three per leg across four legs, plus one for the head), so three of its sixteen channels sit spare — together with an IMU and a battery ADC, all behind a hard ~50 Hz motion deadline.
- **I²C bus 2** carries a single voice-recognition module that clock-stretches and is polled slowly.
- **Three discrete signals**: an addressable LED chain (timing-exact serial), a buzzer, and an ultrasonic range sensor (a one-shot ping and echo).

Nothing about the object set is given. We *derive* it, by walking the procedure.

**Steps 1–2 — enumerate, then triage.** The serialized resources are the two I²C buses and the three discrete pins. Now triage against the smart pins: the WS2812 LED (precise serial framing), the buzzer (tone), and the ultrasonic trig-and-echo (pulse out, pulse measure) each map onto a smart-pin mode that carries the *timing* — so no cog bit-bangs any of them. But timing isn't ownership: each still has to be *serviced* — the LED frame stepped, the tone set, the ping fired and its echo read — and because the smart pins carry the timing jitter-free, the three don't each need a

cog. They collapse onto **one non-blocking I/O cog** that multiplexes all of them, tasked by the top level through a mailbox. The two I²C buses are multi-byte stateful protocols; they survive triage and need software owners — but hold on to the distinction the budget will hold you to: *no smart pin can own the I²C protocol*, yet each bus still rides a **pair of smart pins** (an SCL clock and an SDA data line, configured together for speed and for their position relative to each other), so the two buses consume four smart pins even though a cog drives them. What the physical axis bought here isn't fewer *owners* — it's fewer *cadences*: peripherals that might each have demanded a cog fold onto one low-work cog because the smart pins carry their timing.

**Step 3 — assign owners.** Bus 1 carries three devices — servos (through a PWM chip), the IMU, and the battery ADC — behind one timing budget, so it gets one owning **body-control cog** with a *single shared transport* the three register-level drivers call into. Everything low-work goes onto a second, **I/O cog**: the three smart-pin-timed discretes *and* the voice module, which sits by itself on the second I²C bus. That one cog multiplexes them all non-blocking. Notice the same protocol — I²C — ends up on two different cogs with two different transport shapes, decided entirely by sharing topology and cadence.

**Steps 4–5 — cadences, and the same-bus conflict.** Bus 1 serves three cadences at once: servos near 50 Hz, the IMU near 100 Hz, the battery near 1 Hz. Force 1 won't let us split that bus across cogs, and Force 3 won't let us pretend the cadences are the same. So the resolution is three *cooperative tasks inside the bus-1 cog* — a sense task, a motion task, a slower dispatch task — each running at its own cadence and yielding at bus-transaction boundaries. We also flag one discrete-to-continuous path: a "walk" command has to become a smooth servo trajectory, so a slew engine is going to be needed.

**Step 6 — draw the seams, per plane.** The orchestrator-to-motion seam is a *control-plane* link: a latest-wins command mailbox with a sequence/acknowledge handshake, arguments written first and the sequence counter bumped last, so a torn read is impossible without a lock. Motion-to-everyone is a *data/telemetry* link: lock-free published telemetry — attitude, battery, mode, leg angles — sitting in atomic single longs with one writer and any number of lockless readers. Inbound device events (a finished ping, a recognized word) are an *event-plane* link: a value plus a bumped freshness counter that the slow poll edge-detects. Nothing blocks anywhere — the 50 Hz loop never waits on the orchestrator, and the orchestrator never waits on a device.

**Step 7 — layer the motion branch.** It splits by unit conversion into four tiers: the PWM-chip register driver (changes if the chip changes); then servo pulse-width and channel semantics (changes if the wiring changes); then leg inverse-kinematics, foot-XYZ to joint-degrees (changes if the leg geometry changes); then the gait and pose policy (changes if the behavior changes). A line-count rule would never have produced that stack; the unit-conversion rule produced it on its own.

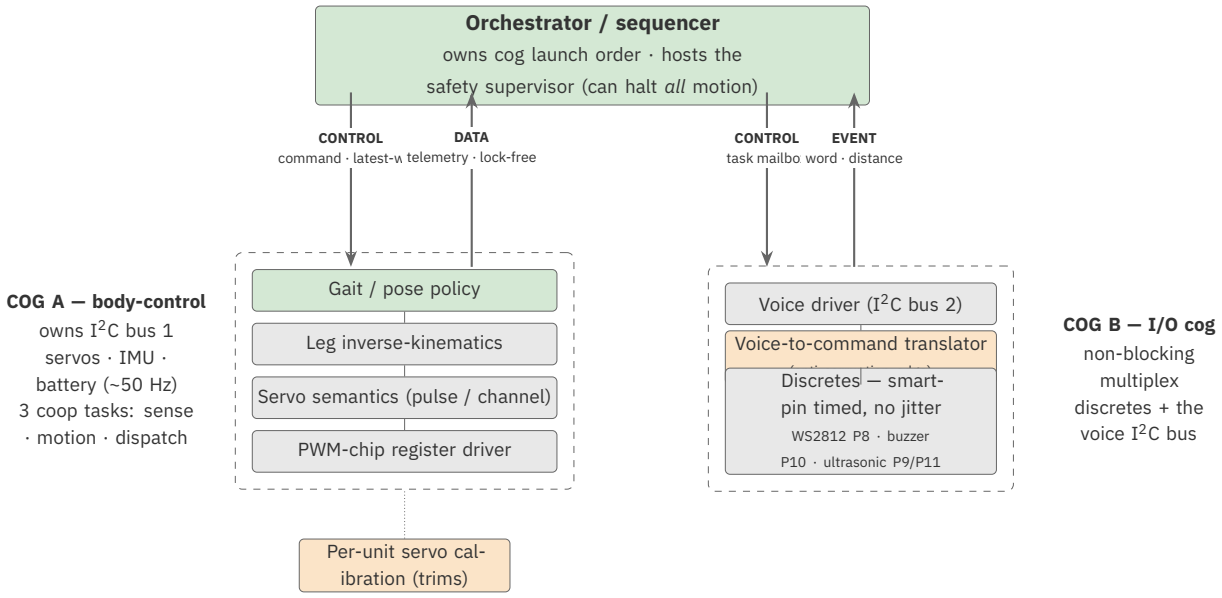
**Step 8 — place the cross-cutting objects.** A critical-battery hard-halt latch sits above policy and suppresses *all* motion regardless of intent (safety). A voice-vocabulary-to-internal-command map sits at the edge, separate from both the recognizer driver and the policy (translation). A per-joint trim store the drivers read but never hard-code (configuration). A bring-up test per layer — bus scan, then chip, then servo-center, then leg IK, then gait (testability). And the top-level

orchestrator owns the launch order and wakes the PWM chip from sleep before any servo write (lifecycle). None of these is a node in the tree; each guards or spans it.

**Step 9 — reconcile against budget and deadline.** Tally the lattice for this application:

| Resource | This application uses | Of the limit |
|---|---|---|
| cogs | orchestrator, body-control (I²C bus 1: servos/IMU/battery), I/O cog (discretes + voice bus 2) — three | 8 |
| Smart pins | discretes P8–P11 (WS2812, buzzer, ultrasonic trig/echo) + two I²C SCL/SDA pairs — about eight | 64 |
| Locks | none — telemetry is single-writer atomic publish | 16 |
| CORDIC | one shared engine, uncontended at this scale | one shared |
| Hub bandwidth | modest — mailbox words, no bulk streaming | egg-beater rotation |

It fits, with cogs to spare, and nothing forces a re-cut. Now judge it: coupling is *low* — telemetry crosses as atomic longs, with no shared invariant and no locks — and the one dynamic change-coupling that crosses a cog boundary (execution order on the command mailbox) was already tamed to static by the publish-last discipline. This is a min-cut.




***An illustration — one application's derived structure, not a template.*** *A different hardware mix yields a different, equally sound shape.*

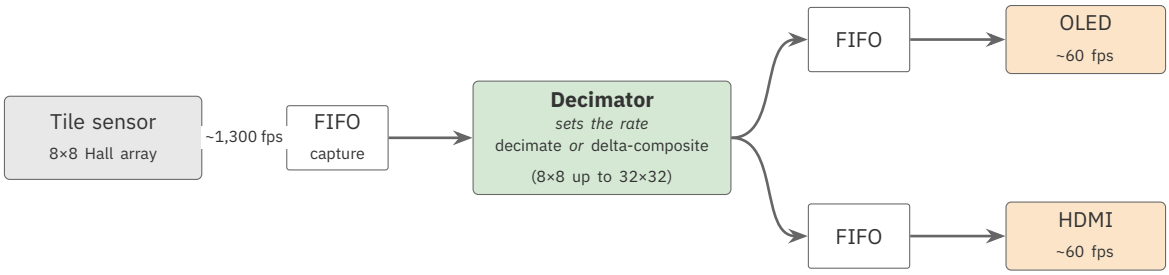
**Figure 9.1.** The object-and-cog map this derivation produced — read it for the moves, not the result. A different hardware mix yields a different, equally sound shape.

Now step back and notice what just happened — and especially what *didn't*. We never started from a parts list and reached for the nearest matching template. We started from the *wires and the timing*, ran the forces in order, and the object set *fell out*. Three things appeared that no catalogue could have handed us: the two I²C transports are the same protocol with different state models, decided by sharing topology; the rate adapters — the in-cog cooperative tasks and the slew engine — correspond to no chip and no feature, they fell out of rate *mismatches*; and the cross-cutting objects had nowhere to live until the tree was drawn, then each took a definite place. Run that same routine on *your* application and you'll get a different object set, equally sound. The shape is the routine's output, not its input.

## A second application, a different answer

The strongest evidence that this is a method and not a catalogue is to watch it produce a *different* answer on a different application — so let's do that, quickly. Swap the robot, a control-plane application that mostly shuffles small command words, for a *data-plane* one: a fast image sensor streaming full frames out to two displays at once. Run the *same* nine steps, and three things come out visibly different from the robot's — not because we changed the method, but because the wiring is different.

First, the producer — the sensor reader — gets its own cog, but *not* because it owns a contested bus. It gets one because any interruption mid-sample corrupts the read: here it's **determinism**, not resource ownership, that forces the cog. That's Force 1 in a shape the robot never showed. Second, the rate adapter is nothing like the robot's. The robot met three cadences on *one shared bus* and answered with cooperative tasks inside the one owning cog. Here the data path is a genuine pipeline: the sensor pours captured frames into a **FIFO**; a **decimator** pulls from that FIFO and *establishes the rate*, bringing the sensor's ~1,300 fps down to the ~60 fps the displays want and choosing *how* as it goes — sometimes plain decimation (drop frames), sometimes *studying the deltas between successive frames and compositing* them on purpose, the jitter-and-delta trick that lifts an 8×8 sensor to an effective 16×16 or 32×32. The decimator then writes into **two more FIFOs, one per display**, and each display runs flat-out, draining its own FIFO and drawing whatever it finds. A FIFO at every stage — sensor to FIFO, FIFO to decimator, decimator to two display FIFOs — not one wire from sensor to screen; the same force as the robot's rate adapter, a genuinely different object. Third, the budget line that binds is no longer cogs — it's **hub bandwidth**, because bulk frames move through the pool and the streamer, and the choice of whether the displays *share* one copy of a frame or each take their own is a bandwidth trade the robot never had to weigh.




*A FIFO at every stage — and the decimator, not the sensor or the displays, sets the rate.*

**Figure 9.2.** The second example's data-plane pipeline: a single tile sensor into a capture FIFO, a single decimator that sets the rate (plain decimation or delta-compositing, up to 32×32), then two peer output paths — a FIFO and a display each, OLED and HDMI. A FIFO at every stage; the rate is the decimator's to set, and the two displays are equals hanging off it.

One nuance from this application's triage step is worth keeping, because it guards against applying a rule too hard. One display is an OLED whose entire frame is a single ~17 ms blocking SPI transfer. A smart pin *can* absorb SPI — the triage flags it as a candidate — and yet here a tight hand-written streaming loop meets the frame budget better, so bit-banging it is the *right* call. The point isn't the answer; it's that "you're bit-banging something a pin could do" is a **signature to check, not a verdict**. You check it, and if you override it you *record why*, along with the one thing that would reverse the decision (here: the day that transfer moves onto the streamer). A named deviation is a decision; an unexamined one is the bug.

Notice what did and didn't carry over from the robot. None of these boundaries did — not the cog map, not the fan-out, not the OLED call. The *procedure* did. That is the entire claim of this part in one line: **carry the method, never the map.**

## Where this leaves you

You came into this part able to write a P2 program. You leave it able to *design* one — to look at an embedded application you've never seen, start from its wires and its deadlines, and derive a sound set of cooperating objects across the fabric, then judge that set against a crisp objective rather than a feeling. That's the skill that keeps you on the spatial side of the line we drew at the start: function spread across the chip, not funnelled back through one core.

A closing word on how to hold all this. The forces, the procedure, and the judging tools are the method; the two worked applications — the robot dog and the streaming pipeline — were only the method made visible, run once each on deliberately different hardware. The method is what you keep.

Start from the wires, run the forces, judge the cut, and let the hardware, not habit, hand you the shape — that is what it means to think in P2. You've now done the front-of-project work of Part I and this decomposition by hand. One part remains, and it changes the cost and the reach of all of it: the same work, walked once more, with an agent at your side.

# Part III — The Same Work, with an Agent

You now know the front of a project (Part I) and how to derive its architecture (Part II). This part walks the *same* work a third time, with an AI agent in the loop, asking one question of each step: *what changes when you have an agent at your side?*

Be clear about what does **not** change. The agent removes none of the judgment. You still decide what to build, you still own the pin map and hold the logic-analyzer probe, and you still judge the cut against the hardest deadline. What the agent changes is the *cost* and the *reach* of each step — and, on the real projects behind this book, that change was large enough to measure in weeks saved and ceilings lifted. This is an additive lens on a process you already understand, not a new process. It begins, though, with a shift in how you *think about the agent itself* — so we start there, then walk the work.

# Chapter 10: The Mindset

## Sufficient Guidance, Not the Perfect Prompt

There is a popular idea that working with an agent is about finding the right *prompt* — the magic phrasing that unlocks a good answer. That is not the mindset this chapter teaches. The question worth asking is never "what words summon the result," it is: *how do I give the agent sufficient guidance to do high-quality work?* When you ask it to generate code, that guidance runs along three dimensions. It needs the **requirements** — what the code must actually do. It needs the **process** — how to go about building it. And it needs a **foundational understanding of the target language** — a real grounding in Spin2 and PASM2, not a guess.

Each of those has a concrete home. The requirements come from documentation the agent itself can produce first — a *theory of operations* for the part or the reference code, which we'll meet in the next chapter. The process comes from **skills**: the procedures you've learned, written down where the agent can follow them. And the foundational language understanding comes from the **P2 Knowledge Base** — the same curated body of architecture, instructions, and Spin2 semantics this guide is drawn from, served to the agent so it writes *correct* P2 code rather than plausible fiction. Supply those three, and you have supplied sufficient guidance. What you add on top of them is the one thing no mechanism provides: **intent** — what you are actually trying to build, and why.

But there is a step between handing over that guidance and letting the agent build, and skipping it is where reliability quietly leaks away. Providing the resources is necessary, not sufficient — you also have to confirm they were *understood*. Before the agent writes anything headed for production, have it **tell you back** what it intends to do: its reading of the requirements, the process it will follow, the P2 mechanisms it means to use. Then **iterate on that understanding until it is actually complete** — correct the misreadings, fill the gaps — and only then let it proceed. A misunderstanding caught here costs a sentence; the same one caught after the code runs costs a debugging session, and the same one that slips into a release costs far more. Success — result meeting expectation — is reached far more reliably by closing the understanding gap *before* the work than by correcting the output after it. Don't let the agent move ahead until everything is understood.

The reason this matters is the shape of the change an agent makes. The right image is not a faster typist; it is an **exoskeleton**. Agentic help is to development time what an exoskeleton is to human strength — it *amplifies*, letting you do things you could not have done on your own, and do them in far less time. And the corollary is important: you amplify what you *already know*, you don't abandon it. You needn't leave your comfort zone or stop doing the things you're good at. The goal is to have more fun and reach farther, keeping your own hard-won knowledge and letting the agent extend it.

Two habits follow from that image, and they run through every chapter ahead. The first is that there is **no single "the agent."** Different tools are built for different work — some are stronger at images, some at broad web research. A question that wants a wide survey of what exists, at what price, is natural to put to a web-based research tool; the build work lives with the agent in your terminal. This is not a rule about which product to use — it's a permission slip: bring whatever agents you're comfortable with, and don't let anyone, including this book, prescribe one. The second habit is that the agent changes *how you arrive at an answer*. A broader search surfaces more possibilities, so the answer you commit to is no longer the first one that worked or the easiest one to reach — it's the **best-researched** one, and you're more convinced of it because you saw what you were choosing among. That is a better way to learn, not just a faster one.

# Chapter 11: Deciding and Learning, with an Agent

This chapter walks the first half of Part I again — deciding what to build, and learning the hardware — with the agent in the loop.

The front of a project is slowest at research, which is exactly where the agent is strongest. Choosing a part is a recurring act: market research, a price point, an honest look at how hard the thing is to talk to. It happens on nearly every sensor and actuator you pick, and an agent collapses it — coming down to a candidate, finding usage examples, even turning up the YouTube video of someone driving it — so the distance from *intent* to *chosen part* shrinks from days to an afternoon. A trainable voice sensor in the fifty-to-a-hundred-dollar range was found exactly this way, by a research tool asked to survey the field by price. Feasibility — the *what's-even-possible* question that should precede design — becomes something you can simply ask: hand an agent the datasheets and a schematic and it reasons out the practical limit before you spend a dollar. The imaging tile's ceiling of roughly thirteen hundred frames a second came back from that kind of question, put to an agent backed by the Knowledge Base. The larger design calls — narrow bus versus broad, or offloading work to a companion device instead of porting it — stay yours; what the agent widens is the set of options you're choosing *among*, and how fast you can price one and rule it out.

Learning the hardware is the single biggest time-sink of Part I, and the agent attacks it directly. Datasheets it reads and summarizes; a datasheet in a language you don't read, it translates — and where the agent itself won't translate, a web browser's translation of the PDF will, and the agent studies the result. A folder of half-working example code in three languages it digests and explains. The move worth naming here is the **theory of operations**: on first contact with an unfamiliar codebase, have the agent inventory it — how many programs, which are libraries, which matter to you — and then write, for each important piece, *what it does, why, and how*, regardless of the language it's in. You come away understanding code you never read a line of, and you can reuse its techniques, translated into Spin2. That same theory-of-operations document, when it captures timing, doubles as an implementation guide the agent builds from — its own output steering its own code toward a more accurate first pass. Where there was no schematic at all, this is how you proceed: the robot dog was lifted off its original Arduino by pointing the agent at the vendor's code and reverse-engineering every sensor and actuator's communication together.

Your instruments become partners in this chapter, too. The logic analyzer used to come with a hand-kept journal — which probe, which cable color, mapped to which pin — verified by eye. Now you speak the hookup and the agent **annotates the code** with it, so the wiring map and the code live together and stay correct. Show it a capture that looks wrong and it will propose *why* — a miswire, a connection to recheck — turning the analyzer into a troubleshooting partner, and the same holds for a scope or any external instrument. It will even guide the physical hookup it cannot perform: describe the rig and the pins you have, and it will walk you through wiring it step by step and hand you the mapping document, though your hands still hold the wire.

Which is the boundary worth stating plainly. The agent can read every datasheet in the world and still cannot hold the probe, bolt the sensor to its fixture, or see that a mechanism limits a servo's travel. **You are its senses and hands in the physical world**, and the constraints only you can observe are yours to supply — the range of travel a mounted servo actually has (so it constrains motion and the arm doesn't destroy itself under test), the repeatability the hardware can really deliver, the wiring, what the fixture allows. These are requirements in the same sense as any other; they're just the subclass that comes from a human at the bench. And one more piece of guidance is uniquely yours to give: the **frames of reference**. The complex, rework-prone code — rotation, daisy-chain topology, pixel mapping — becomes tractable the moment you name the perspectives it must hold. Tell the agent that a string of panels is one logical display, that the wiring enters at a given point, that the panel order may vary and these constants describe it, that the panels need an initialization chain — and it will derive the wiring, the bit ordering, the init sequence, and the replication correctly. Naming the frame is the human's real contribution; within it, the agent works.

# Chapter 12: Building and Shipping, with an Agent

Now the second half of Part I — building the capability, and finishing the job.

The in-head translation that used to define the building phase — carrying a C or Arduino idea across into Spin2 one line at a time — the agent does with you at a different speed, and the tables and boilerplate you once hand-generated, it generates. But the deeper changes are three. When the hardware itself changes shape — moving a servo *behind* a PWM-generation chip, so the servo object now speaks to the chip that drives the servo — you describe the new indirection and the agent **reshapes the code**; what used to be a re-engineering slog becomes a change you recover from in an afternoon, back on the air where you left off. When performance is the goal, the division of labor is clean: *you* decide where speed matters and why; the agent helps decide *how* to reach it with the P2's own resources — LUT RAM, the CORDIC, the streamer — and external PSRAM. And when you build a piece in the middle — a FIFO, say — you build it as a **standalone object with an agent-written regression test**, proven before it's wired in, so that by the time it's inside the whole application it's a *tested component* and no longer a suspect when something breaks.

There's a deeper shift underneath all of this. A **hosted** agent that holds the whole P2 toolchain — the `pnut-ts` compiler, the `pnut-term-ts` terminal-and-debug host, and the P2 Knowledge Base on tap — can close the *entire* loop by itself: write the code, compile it, download it to a real P2, run it, read the `DEBUG` output and logs that come back, and go around again. It isn't drafting code for you to run and report back on; it is running its own experiment on real silicon and reading its own result, round-trip after round-trip. That autonomy is what lets the isolation tests above actually get written *and passed* without you in the loop for every cycle — and it's the line between an agent that merely *suggests* and one that *converges*: you set the target and the check that says "done," and it iterates until it reaches it. The judgment of what "done" means stays yours; the grind of getting there does not.

The single largest change, though, is the one Part I foreshadowed: the ceiling that used to be *yours* rather than the chip's. A six-axis arm once stalled at the edge of one engineer's comfort with the mathematics of inverse kinematics — the code was reachable, the math wasn't. With an agent that ceiling lifts. The same arm can carry a full inverse-kinematics solution and coordinated motion that brings the whole arm to its target in the least time rather than driving each joint in turn; it can even plan a move so the center of gravity shifts least and the arm stays stable — math that was simply out of reach before. That is the clearest single thing an agent changes: not the work you could already do faster, but the work you *couldn't do at all*. Taste still leads, all the same — what an interface should feel like, how a thing should behave, stays a human call the agent serves rather than makes.

Finishing changes too, in two places. The documentation — the usable driver docs, the write-ups, the examples — the agent drafts, turning the closing ritual from a chore into a review; and if you tell

it which documents do what, why, and when each must be updated, it will keep the whole catalog, and a changelog of what changed each pass, consistent for you. When a one-off deserves to become a reusable part, the agent recognizes the standalone pieces, suggests publishing them to OBEX, and — because it's grounded in the Knowledge Base — enriches the public documentation with the P2 techniques a reader should know but you didn't think to mention. And the long tail, the most demoralizing part of Part I, is where the agent earns its keep most surprisingly. A vendor ships new code and the year-long stall of reconciling it becomes surgical: take your original source, diff the new release, apply just the meaningful changes, pull the new binaries, and run — days instead of weeks, which is why a stalled time-of-flight project becomes worth reviving at all. The same move, pointed at *your own* history, finds a performance regression a user reports between an old release and a new one; pointed at the *Knowledge Base* as it improves, it re-audits old code against better examples and fixed bugs to unlock speed that wasn't reachable when you first shipped.

# Chapter 13: Through the Decomposition, with an Agent

The middle of the book — the decomposition itself — changes as well, and this is where an agent is most easily misunderstood. It can help you *postulate* a decomposition and build the layers under it: the robot dog was reverse-engineered, decomposed onto the P2, and coded with agent help in a week or two rather than a month or two. It can run the first-contact procedure alongside you, sanity-check a proposed cut against the four forces, and — because you've named the planes — catch a data-plane concern smuggled into the control plane before it ships. Naming the frames of reference, the habit from Chapter 11, is exactly what lets it reason about a decomposition without losing the thread.

What the agent does *not* do is own the reconciliation. The forces still argue, the hardest deadline still wins, and the final CALL — which cog owns what, where the seam goes, what adapts between cadences — is still yours. The agent is a fast, well-read partner in that judgment, one that has the whole Knowledge Base at hand and can enumerate the two or three techniques the P2 offers for a given problem in seconds. It is not a substitute for the judgment itself.

# Chapter 14: New Reach

## Beyond What You Could Build Alone

Walk back over the three parts and the pattern is plain: every phase got cheaper or reached farther, and not one got removed. You still choose what to build, still own the pin map and the probe, still judge the cut. But the change worth ending on is not the speed — it's the **reach**. The most important thing an agent changes is not the work you could already do, done faster; it's the work you *couldn't do at all* coming within reach.

That reach shows up as new ceilings lifted, one after another. The math ceiling — inverse kinematics, a vision system that finds an object and places it — we've already met. There is also a *platform* ceiling: standing up the Raspberry-Pi and Linux side of a gateway — which packages to install, how to configure a web server, how to pull high-precision time, how to talk to external APIs — is second nature to someone with years of Linux and a wall to someone without. The agent helps the newcomer understand what facilities exist, why to choose each, and then installs the ones they pick; the platform they'd never touched becomes crossable. And there is a whole class of artifact that used to demand a second discipline entirely: a **mobile control panel** for your embedded application. Add a Bluetooth Low Energy link — a couple of wires — and the agent will handle the characteristics to expose, the GATT, the device-side implementation, and the phone app itself, on Android or iOS. What once required *being* a mobile developer is now within reach of any embedded developer with agentic help; you supply the intent and, on iOS, the tooling license.

Notice, too, that the same tool reads differently to different people. To an expert it is amplification — more of what they already do well, faster and broader. To a newcomer it is passage onto ground they'd never have crossed alone. Both are the exoskeleton; they only differ in where your own feet already stood. And amplification compounds: once the agent makes a hard core tractable — the arm's motion, a complex high-speed sensor's driver — you can *compose*, dropping a tested, exported object with its theory of operations and its logic-analyzer-proven communications straight onto the next project, so a 180-degree field-of-view sensor lands on the walking robot as a near-afterthought. Each finished piece becomes raw material for the next, and the reach compounds from one project to the next.

# In Closing

Look back at the distance covered. You began with a real project and the ordinary, unglamorous front of the work — deciding what to build, learning parts nobody documented well, wiring them, proving they talk, making them fast, and shipping them so someone else could pick them up. Then you took that wired-up, understood application and *derived* its shape instead of guessing it — reading the forces the hardware and the deadlines press on you, and letting them, not habit, hand you which cog owns what. And then you walked all of it a third time with an agent beside you, and watched every phase get cheaper or reach farther — with a few designs that weren't reachable at all before coming into reach.

What you carry out of these pages is not a set of answers; it's a *method*. Point it at hardware you've never seen — a new sensor, an unfamiliar bus, a tighter deadline — and it will hand you a sound design for *that* application, different from every example here and correct for its own reasons. That is the difference between a catalogue and a craft: a catalogue runs out at the edge of what it listed; a craft doesn't run out.

There's a symmetry worth ending on. The knowledge an agent draws on to help you think in P2 — the architecture, the instructions, the very decomposition reasoning of Part II — is the same curated body this guide was written from. The better that knowledge grows, the better a partner it makes. You are, in a sense, holding one end of it, and the community is writing the other.

So go build something. Start from the wires, run the forces, judge the cut — and let the agent carry the load it carries well, while you keep the judgment that was always yours. The rest of these pages are the shelf behind you: Appendix A on why we borrow the language of FPGAs, Appendix B for the literature beneath the method, a glossary for the terms, and a map into the reference manuals for every part you'll actually program. Reach for them when a job calls. Otherwise the P2 is waiting — eight cogs, sixty-four pins, and a handful of forces you now know how to read. Go think in it.

# Appendix A: Computing in Space and Time (Why We Borrow FPGA Language)

Throughout this guide — and especially in Part II — we describe the P2 with words borrowed from the world of FPGAs and hardware design: *spatial*, *fabric*, *pipeline*, *dataflow*, *back-pressure*, *systolic*. The borrowing is deliberate and useful, but it carries a risk: taken too literally, those words would say the P2 *is* an FPGA, and it isn't. This appendix sets the record straight — what the FPGA vocabulary buys us, and exactly where it stops.

## The temporal-to-spatial spectrum

Computation can be placed on a spectrum by *how* a machine does many things at once. At one end is the purely **temporal** machine: a single processor core executing one instruction stream, doing more only by running faster or by time-slicing that one core. At the other end is the purely **spatial** machine: an FPGA, where function is laid out as physical parallel hardware — many circuits computing simultaneously, configured by a synthesis tool, with no instruction stream at all.

The P2 sits between these poles, nearer the spatial end than a conventional microcontroller but well short of an FPGA. Its eight deterministic cogs and sixty-four programmable smart pins are real, parallel computing elements you assign function to — that is the spatial character. But each element runs *software*, an instruction stream of its own — that is the temporal character it never sheds. The phrase the guide uses, **coarse-grained spatial fabric**, names exactly this in-between position: spatial in how you allocate function, temporal in how each element actually computes.

## What transfers, and what doesn't

The FPGA *mindset* transfers; the FPGA *claims* do not. Three honest qualifications keep the borrowing safe:

- **The P2 is coarse-grained, not fine-grained.** An FPGA's fabric is a sea of logic gates and routing you configure at the bit level. The P2's "fabric" is a handful of full 32-bit processors and some smart pins. You allocate whole cogs to jobs; you do not wire gates. This is a difference of *kind*, not degree.
- **The P2 is still software.** You write programs and launch cogs. There is no hardware-description language, no logic synthesis, and crucially **no place-and-route** — the step that maps an FPGA design onto physical silicon has no P2 equivalent. The determinism a cog gives you comes from fixed instruction timing, not from synthesized circuitry.
- **We borrow the discipline, not the identity.** "Think spatially" means *assign one sustained job per element and let it run* — a design discipline. It does not mean the P2 reconfigures its

silicon. Every spatial behavior on the P2 is something you *arrange in software*, which is also why a sloppy decomposition can throw it away (the whole argument of Part II).

Hold those three in mind and the vocabulary is a gift: it imports decades of hardware-design reasoning about pipelines, latency, and dataflow into a software setting where it genuinely applies. Forget them, and the same words mislead.

## The terminology, mapped

Each borrowed term is pinned below to its FPGA-world meaning, its P2 mapping, and — the column that does the guarding — where the mapping goes loose. First, the vocabulary for *laying computation out* as parallel hardware:

| Term | In the FPGA / hardware world | On the P2 | Where the mapping is loose |
|---|---|---|---|
| Spatial computing | Function laid out as physical parallel circuitry | Function assigned across cogs and smart pins, each running one job continuously | The P2 runs instruction streams; "spatial" is the *allocation* pattern, not literal gates |
| Fabric | The sea of configurable logic blocks and routing | The 8 cogs + 64 smart pins + the hub interconnect you allocate onto | The P2 fabric is a few coarse elements, not a fine-grained gate array |
| Coarse-grained | Processing elements larger than a single gate | Each element is a whole 32-bit processor or a smart pin | This is the defining gap — the P2 is far coarser than even a coarse-grained array |
| Pipeline | Data through chained hardware stages, throughput set by the clock | Data through a chain of cogs, throughput set by the pipeline rate, not instruction count | Each cog stage runs software with its own latency; stages are not register-locked like hardware |
| Dataflow | Computation driven by data availability along channels | cogs exchanging data through hub channels and mailboxes; correctness by data order | There is no hardware firing rule; the dataflow discipline is something you implement |
| Systolic array | A regular array of cells rhythmically passing data to neighbors | cogs as pipeline stages handing data along, sometimes via adjacent-cog LUT sharing | Only adjacent cog pairs share a LUT; it is a small, irregular array, not a large regular mesh |

Then the vocabulary for the *resources, timing, and dataflow* of the machine — ending with the one term that does not cross over at all:

| Term | In the FPGA / hardware world | On the P2 | Where the mapping is loose |
|---|---|---|---|
| Resource lattice | (Loosely) the fixed grid of resources a design maps onto | The finite, heterogeneous set you budget against: 8 cogs, 64 smart pins, 1 CORDIC, 16 locks, hub bandwidth, LUT pairs | "Lattice" here means a fixed resource budget, not FPGA routing |
| Back-pressure | A downstream consumer signalling it cannot keep up, throttling upstream | A slow consumer forcing a fast producer to wait at a seam; managed with buffers and the hub FIFO | Same concept, implemented in software at hub seams |
| Latency / throughput | Time through a stage; rate of completed items | The same, measured against the system clock and the egg-beater rotation | Transfers cleanly — this pair means the same on both sides |
| Latency-insensitive | Design so correctness depends on data order, not arrival time | Hub channels designed so hub jitter is harmless by construction | A discipline you adopt, not a property the silicon enforces for you |
| GALS (globally asynchronous, locally synchronous) | Synchronous islands joined by an asynchronous interconnect | Locally-synchronous, deterministic cogs joined by the asynchronous hub fabric | An exact characterization — this one transfers well |
| Place-and-route | The synthesis step mapping a design onto physical gates and wires | *(no equivalent)* — you write software and launch cogs; nothing is synthesized | The sharpest "does not transfer": there is no P2 place-and-route at all |

The last row is the one to remember. The P2 borrows the FPGA's *way of thinking about parallel work* while remaining, start to finish, a software machine. Appendix B points you to the literature behind both halves of that sentence.

# Appendix B: Further Reading on Functional Decomposition

Part II's method rests on a body of published work older and deeper than the P2 itself. This is the short list — each entry with a line on why it matters here. It runs along the two axes Part II used: the **logical** axis (how to cut software well, independent of any chip) and the **physical and concurrent** axis (how parallel, communicating elements compute — the literature closest to what the P2 actually is). A third short group covers boundaries, real-time scheduling, and the generative stance the whole approach takes.

## The logical axis — cohesion, coupling, and what to hide

- **Parnas, D.L. — "On the Criteria To Be Used in Decomposing Systems into Modules."** ***Communications of the ACM*****, vol. 15, no. 12, 1972, pp. 1053–1058.** The origin of *information hiding*: decompose around the decisions likely to change, not around processing steps. This is the principle under Force 4 (layer by axis of change).
- **Constantine, L.L. & Yourdon, E. —** ***Structured Design: Fundamentals of a Discipline of Computer Program and Systems Design.*** **Prentice-Hall, 1979.** Where *coupling* and *cohesion* come from — the measures behind a good seam: low coupling across cogs, high cohesion within one.
- **Page-Jones, M. —** ***Fundamentals of Object-Oriented Design in UML.*** **Addison-Wesley, 1999.** Its treatment of *connascence* (this guide's **change-coupling**) is the sharpest tool in Chapter 8's "judging the cut" section — and the source of the static-versus-dynamic distinction that, on the P2, separates a safe seam from a race.

## The physical and concurrent axis — communicating processes and dataflow

- **Hoare, C.A.R. — "Communicating Sequential Processes."** ***Communications of the ACM*****, vol. 21, no. 8, 1978, pp. 666–677; expanded as** ***Communicating Sequential Processes*****, Prentice-Hall, 1985.** The formal model in which cogs are processes and mailboxes are channels. If one work explains why the P2's no-shared-OS, message-passing shape is sound, it is this one.
- **INMOS Ltd. —** ***occam Programming Manual.*** **Prentice-Hall, 1984.** The Transputer's language — independent processors, a message-passing fabric, no shared operating system. The P2 is very nearly a Transputer reborn, and inherits its decades of correctness reasoning.
- **Kahn, G. — "The Semantics of a Simple Language for Parallel Programming."** ***Proceedings of the IFIP Congress 74*****, Stockholm, 1974, pp. 471–475.** Kahn process networks:

processes that communicate only by blocking reads on FIFO channels are *determinate regardless of timing* — the rule that makes inter-cog dataflow survive hub jitter.

- **Kung, H.T. & Leiserson, C.E. — "Systolic Arrays (for VLSI)." In Mead, C. & Conway, L., *Introduction to VLSI Systems*, Addison-Wesley, 1980 (§8.3).** Rhythmic data passing through a regular array of processing elements — the mental model for using cogs as pipeline stages.
- **Lee, E.A. & Messerschmitt, D.G. — "Synchronous Data Flow." *Proceedings of the IEEE*, vol. 75, no. 9, 1987, pp. 1235–1245.** Static data rates yield computable buffer sizes — the math behind Force 3's rate adapters and the sizing of a buffer.
- **Carloni, L.P., McMillan, K.L. & Sangiovanni-Vincentelli, A.L. — "Theory of Latency-Insensitive Design." *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 20, no. 9, 2001, pp. 1059–1076.** Correctness by data *order*, not arrival *time* — the formal bridge to the spatial domain and the discipline that makes hub jitter harmless.
- **Chapiro, D.M. — *Globally-Asynchronous Locally-Synchronous Systems.* PhD thesis, Stanford University, 1984.** The exact characterization of the P2 — locally synchronous cogs, an asynchronous hub fabric — and the source of the clock-domain-crossing discipline Force 3 borrows.

## Boundaries, real-time, and the generative stance

- **Evans, E. — *Domain-Driven Design: Tackling Complexity in the Heart of Software.* Addison-Wesley, 2003.** *Bounded contexts* as subsystem boundaries with their own internal language — the reasoning behind the external-interface translator (cross-cutting force C2).
- **Liu, C.L. & Layland, J.W. — "Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment." *Journal of the ACM*, vol. 20, no. 1, 1973, pp. 46–61.** Rate-monotonic scheduling — assigning urgency by cadence and reasoning about deadlines, the theory under the event plane's latency tiers.
- **Alexander, C. — *A Pattern Language: Towns, Buildings, Construction.* Oxford University Press, 1977.** The source of the idea that patterns should *compose into a grammar* rather than sit in a catalogue — exactly the stance Part II takes toward decomposition: a method that generates, not a set of templates to copy.

# Glossary

Terms as this guide uses them, weighted toward the decomposition vocabulary of Part II. For the silicon parts themselves — cog, hub, smart pin, CORDIC, streamer — see *Getting Started*.

**Altitude layering (Force 4).** The vertical decomposition force: within one ownership domain, stack objects so each tier does exactly one unit conversion and changes for exactly one reason — bits, then registers, then physical units, then behavior.

**Back-pressure.** The resistance a seam imposes when a consumer cannot keep up with a producer; measured as the change-coupling crossing the seam times the cost of the channel that carries it. A good boundary minimizes it.

**Coarse-grained spatial fabric.** The P2 seen as a modest number of real parallel computing elements (8 cogs, 64 smart pins) you assign function to — spatial in allocation, but built from whole processors rather than logic gates.

**Cohesion.** How well the parts inside one object belong together. High cohesion within an object is the goal; it is the complement of coupling.

**Change-coupling (connascence).** The relationship by which changing one element forces a change in another to stay correct. *Static* forms are visible in source (name, type, field order); *dynamic* forms are true only at run time (execution order, timing, value). On the P2, dynamic change-coupling crossing a cog boundary shows up as jitter and races.

**Cooperative tasking (tasks-in-a-cog).** Several routines sharing one cog and one bus, each running at its own cadence and yielding at safe points — the resolution when one bus must serve several cadences (Force 1 against Force 3).

**Coupling.** How much crosses between two objects — on the P2, a countable integer: longs per unit time, fields under a shared invariant, locks held across the cut. Minimize it.

**Cross-cutting forces (C1–C5).** The concerns that place objects spanning or guarding the structural tree rather than sitting in it: safety override, external-interface translation, per-unit configuration, testability seam, and lifecycle/init order.

**Data / control / event planes.** The three superimposed relationships in any inter-cog seam — bulk movement (data), commands and state (control), signalling and urgency (event) — each wanting its own mechanism.

**Data-flow contract (Force 2).** The promise a seam makes: blocking CALL, latest-wins mailbox, ring buffer, request/response, or published telemetry. The contract sets the dependency direction and helps place the boundary.

**Flat device list.** The failure mode Force 1 prevents: every chip a sibling driver under `main()`, the shape chosen by analogy rather than derived from the wiring. It compiles, then fails as flaky hardware the moment two cogs touch one resource.

**Funnel.** A “smell”: all data routed through one cog, rebuilding a sequential bottleneck whose loop rate caps the whole system.

**GALS (globally asynchronous, locally synchronous).** The exact shape of the P2 — deterministic, locally-synchronous cogs joined by an asynchronous hub fabric — and the reason cadence crossings need deliberate handling.

**Latency-insensitive.** A channel designed so correctness depends on the order data arrives, not the time — making hub jitter harmless by construction.

**Min-cut.** The objective for a good boundary: draw it where the cohesion gained inside each piece exceeds the back-pressure across the cut.

**Pipeline.** A chain of cogs through which data flows stage to stage; throughput is set by the pipeline’s rate, not by any one stage’s instruction count.

**Publish-last.** The discipline of writing a multi-field update’s payload first and bumping its signalling counter last, so a reader can never catch a torn value — a lockless hand-off made safe by single-long atomicity.

**Rate adaptation (Force 3).** The force that inserts objects wherever two cadences meet: samplers/buffers at rate-domain crossings, and slew/easing engines where a discrete intent must become a continuous stream.

**Resource budget.** The allocation table — cogs, smart pins, locks, CORDIC, hub bandwidth, LUT pairs, cog RAM — kept as a design artifact. “Running out of cogs” on it means the design is too coupled.

**Resource lattice.** The finite, heterogeneous set of P2 resources a design allocates onto; the physical axis of decomposition.

**Resource ownership (Force 1).** The correctness force: each serialized, stateful resource gets exactly one owning cog, with the object boundary tracing the wire.

**Singleton vs. instance transport.** The transport’s state model, decided by sharing topology — a shared singleton when several devices share one bus, a self-contained instance when a device is alone on its bus.

**Slew / easing engine.** The object that turns a discrete command (a “step”) into a smooth, rate-limited trajectory at a device’s native frame rate.

**Spatial computing.** Doing many things at once by laying function out across hardware elements rather than time-slicing one core; on the P2, the discipline of assigning one sustained job per cog or smart pin.

**Systolic array.** A regular arrangement of processing elements that rhythmically pass data to their neighbors — the FPGA-world model behind using cogs as pipeline stages.

**Transport (object).** The single owning object for a bus or serialized resource; the lowest tier of a device stack, the one that speaks bits on the wire.

# Where to Next

This guide is the orientation layer; the reference manuals are where you go for depth. Here is the map.

- **To write the high-level language** — the *Spin2 Reference Manual* (current revision v55): the full object model, every built-in method and operator, the language's syntax in complete detail.
- **To write assembly** — the *P2 Assembly Language Reference*: the PASM2 instruction set, the execution pipeline, cog start/stop, and the inter-cog coordination primitives (locks, atomic access, cog attention) that Part II's seams are built from. For a gentler, tutorial-style on-ramp to PASM2, the *DeSilva PASM2 Tutorial* teaches the assembly language from the ground up.
- **For I/O** — the *P2 I/O & Smart Pins User Guide*: every smart-pin mode, with examples — your first stop whenever a protocol might be absorbable at the pin (Part II's smart-pin triage).
- **For high-speed data** — the *P2 Streamer Programming Guide*: the streamer in full, including the video (VGA, HDMI, composite), audio, and capture modes.
- **For debugging and bring-up** — the *P2 Debug Window Manual* and the *P2 Single-Step Debugger Manual*: the on-chip DEBUG output windows and the single-step debugger, the tools behind Part II's per-layer bring-up tests (cross-cutting force C4).
- **For the silicon itself** — the *Parallax Propeller 2 Documentation v35 - Rev B/C*: the foundational reference — CORDIC operations, the event system, boot sources, and the hardware-timing details the other manuals build on.
- **For the decomposition theory in full** — Appendix A (the spatial/temporal framing) and Appendix B (the reading list behind every force, plane, and judgment tool): the published canon Part II's method rests on, in more depth than a single part can carry.

That is the library. Start where your current job points you, and let the picture, the language, and the working shape you brought from *Getting Started*, and the method from Part II, guide how you put the pieces together.